СЕРВИСНЫЙ ПУЛЬТ ОПЕРАТОРА: ЗАМОК ЯЧЕЙКИ ХРАНЕНИЯ, ДВИГАТЕЛИ, АКБ, СТАТИСТИКА

> **Документ:** docs/SERVICE_CONSOLE.md **Объект:** новое окно веб-пульта gui/ автономного робота-курьера 4WIS/4WID **Версия:** 1.2 · **Дата:** 2026-10 **Статус:** реализовано и проверено; результаты прогонов — §12.5 **Связанные документы:** README.md · GUI.md · SOFTWARE_ARCHITECTURE.md · SERIAL_PROTOCOL.md · ELECTRICAL_AND_PINOUTS.md · EXPLANATORY_NOTE_AVTOVAZ.md

Документ является **техническим планом** (ТЗ + проектная записка) окна «Сервис» и одновременно описанием реализованного решения. Структура: разделы 1–4 — постановка и требования, 5–11 — архитектура и алгоритмы, 12 — тест-план, 13 — соответствие регламенту, 14 — риски и развитие, 15 — порядок работ и критерии приёмки.

---

1. Назначение окна

Окно **«Сервис»** (`view-console`) — единая точка обслуживания робота оператором смены. Оно закрывает четыре задачи, продиктованные регламентом конкурсного задания № 2 и эксплуатацией изделия:

| № | Задача | Регламент (задание № 2) |

| :-: | :--- | :--- |

| 1 | **Доступ к грузу**: ввод пароля, открытие/закрытие ячейки хранения (грузового отсека), журнал доступа | «груз … защищен от несанкционированного доступа», «защита груза от посторонних лиц» (критерии 6, 9) |

| 2 | **Статистика**: рейсы, пробег, энергия, груз, события (знаки, светофор, объезды), выгрузка отчёта | «отчётность», «сбор, хранение и управление диагностическими данными», «информирование о статусе задания» (критерий 13) |

| 3 | **Двигатели**: под каждый из 8 двигателей (4 рулевых NEMA 23 + 4 тяговых BLDC) — поле ввода и кнопка «Ввод» для ручной подачи команды, квитирование, лог строки протокола | «безопасное движение», «готовность к развитию», «программно-аппаратный комплекс» (критерии 7, 8) |

| 4 | **Состояние аккумулятора**: пак 12S3P LiFePO4 — напряжение, SOC, ток, мощность, остаток ёмкости, запас хода, пороги защиты, телеметрия по модулям | «встроенный источник питания», «безопасность» (критерии 7, 8) |

Окно не заменяет существующие вкладки «Пульт», «Ходовая», «Миссия», «Сенсоры», «Безопасность», «Журнал» — оно дополняет их эксплуатационным (сервисным) контуром.

---

2. Требования к решению

2.1. Функциональные требования (ФТ)

ФТ-1. Ячейка хранения (замок):

- ФТ-1.1 поле ввода пароля (маскированный ввод, цифровая клавиатура, Enter);
- ФТ-1.2 проверка пароля, открытие/закрытие замка, индикация состояния «ЗАКРЫТО / ОТКРЫТО / БЛОКИРОВКА»;
- ФТ-1.3 смена пароля (текущий + новый + повтор);
- ФТ-1.4 блокировка ввода после 5 неудачных попыток на 30 с с обратным отсчётом;

- ФТ-1.5 журнал доступа (кто/когда/результат), устойчивый к подделке (хеш-цепочка записей);
- ФТ-1.6 альтернативные каналы: «QR» и «RFID» (имитация считывания);
- ФТ-1.7 предупреждение при открытии вне площадки загрузки/выгрузки, если в отсеке есть груз;
- ФТ-1.8 пароль не хранится в открытом виде (SHA-256 + соль), значения по умолчанию документированы для судей (заводской PIN 2580).

ФТ-2. Статистика:

- ФТ-2.1 живые показатели сессии: пробег, энергия (Вт·ч), расход (Вт·ч/км), средняя/максимальная скорость, время в движении, груз;
- ФТ-2.2 накопительные показатели: рейсы, пробег, энергия, перевезённый груз, события (знаки по типам, светофор, объезды, ручные вмешательства);
- ФТ-2.3 таблица последних рейсов (время, маршрут, дистанция, груз, энергия, результат);
- ФТ-2.4 графики: энергия по рейсам и история SOC/скорости за сессию;
- ФТ-2.5 экспорт: CSV (для таблиц), JSON (машинный отчёт), печатный отчёт (HTML → печать/PDF браузера);
- ФТ-2.6 персистентность в `localStorage` с версионированием схемы;
- ФТ-2.7 сброс статистики с подтверждением.

ФТ-3. Двигатели:

- ФТ-3.1 4 карточки модулей (FL/FR/RL/RR), в каждой два канала: рулевой (NEMA 23, угол в градусах) и тяговый (BLDC Xiaomi M365 Pro, ШИМ/мощность);
- ФТ-3.2 в каждом канале: числовое поле ввода + кнопка **«Ввод»**, а также быстрые кнопки-«джоги» (шаг ±5°, ±45°, тяга −20 % / 0 / +20 %);
- ФТ-3.3 квитирование: целевое значение → текстовая команда fw25 (`w`, `v`, `t`) → HEX в журнале → применение к модели;
- ФТ-3.4 отображение факта: угол, ошибка, шаги, «хоминг», статус, для тяги — об/мин, м/с, ток, температура, напряжение модуля;
- ФТ-3.5 сервисный режим «стенд»: авто/миссия/телеуправление блокируются, робот стоит, вращаются только модули; выход из режима — явной кнопкой;
- ФТ-3.6 интерлоки: ввод запрещён при `E-ST0P` и при активном авторежиме (предлагается сначала остановить миссию);
- ФТ-3.7 групповые операции: «Стоп все», «Момент вкл/выкл», «Хоминг всех», «Синхронный угол» (задать одинаковый угол всем модулям).

ФТ-4. Аккумулятор:

- ФТ-4.1 модель 12S3P LiFePO4 (36 банок 32700): 30.0–43.8 В, 18.6 А·ч, 714.2 Вт·ч;
- ФТ-4.2 расчёт SOC по OCV-кривой LiFePO4 с учётом просадки под током;
- ФТ-4.3 индикация: напряжение, ток, мощность, остаток Вт·ч, запас хода, температура, состояние (разряд/заряд/покой);
- ФТ-4.4 пороги из `config/safety.yaml`: 35.5 В — ползучий режим, 33.5 В — аварийный стоп; отображение текущего уровня риска;
- ФТ-4.5 балансировка: min/max/Δ по 12 группам ячеек (модель);
- ФТ-4.6 телеметрия по 4 модулям: напряжение, ток, температура, вклад в расход;
- ФТ-4.7 история SOC за сессию (график) и счётчик циклов заряда;
- ФТ-4.8 быстрые действия: «На базу · зарядиться» (существующий сценарий), задание целевого процента.

2.2. Нефункциональные требования (НФТ)

Код	Требование
НФТ-1	Никаких внешних зависимостей и сборщиков: чистые `HTML/CSS/JS`, работа по `file://`
НФТ-2	Совместимость с существующим пультом: тема ZMK Vision (тёмный сайдбар `#101b17`, акцент `#d5ff45`), светлая/тёмная темы
НФТ-3	Единый главный цикл `requestAnimationFrame`; тяжёлые перерисовки — не чаще 4 Гц и только для видимой вкладки
НФТ-4	Логика, критичная для корректности (протокол, модель АКБ, пароли, статистика), вынесена в отдельный модуль `gui/console-core.js` и покрыта Node-тестами
НФТ-5	Отказ `localStorage` (приватный режим) не должен ломать пульт: деградация до памяти сессии
НФТ-6	Русскоязычный интерфейс, единицы СИ, формат времени `чч:мм:сс`
НФТ-7	Адаптивность: 2 колонки → 1 колонка при ширине < 1100 px

---

3. Пользовательские сценарии

С-1. Выдача груза получателю (основной). Робот прибыл на площадку Б → оператор открывает окно «Сервис» → вводит PIN (или QR/RFID) → замок открывается → оператор забирает груз → нажимает «Груз сдан» (существующая кнопка подтверждения) → статистика фиксирует рейс → замок закрывается автоматически через 30 с или кнопкой «Закрыть».

С-2. Загрузка на площадке А. Оператор вводит PIN, открывает отсек, укладывает груз, закрывает замок. Система фиксирует массу (ползунок «груз») и запрещает старт миссии с открытым замком.

С-3. Стендовая отладка модуля. Инженер останавливает миссию, включает «Сервисный режим · стенд», в карточке модуля FL задаёт угол 45 и нажимает «Ввод» — рулевой модуль поворачивается, в журнале появляется строка команды `w FL 45.0` (протокол fw25), в карточке видны фактический угол и ошибка. Аналогично подаётся тяга `v 20 f` и контролируются `sc_rpm`, ток и температура.

С-4. Диагностика АКБ. Оператор видит напряжение 35.2 В, уровень «ПОЛЗУЧИЙ», остаток 8 %, запас хода 6 км → нажимает «На базу · зарядиться», задаёт цель 100 %.

С-5. Отчётность. По итогам смены оператор нажимает «Отчёт (печать)» — формируется печатная форма: сводка, рейсы, события, состояние АКБ; либо выгружает CSV/JSON для дальнейшей обработки.

---

4. Термины и обозначения

Термин	Значение
Модуль (pod)	Поворотно-тяговый узел: NEMA 23 (руль) + мотор-колесо Xiaomi M365 Pro (тяга)
Канал	Один двигатель модуля: `steer` или `traction`
ШИМ / PWM	Задание тяги в протоколе: −1000…+1000 (знак — направление)
SOC	State of Charge, оценка заряженности АКБ, %
OCV	Напряжение холостого хода (по нему строится кривая SOC)
Стенд	Сервисный режим: робот обездвижен, модули управляются вручную
Рейс	Интервал автономного движения от старта миссии до прибытия/сдачи груза
Аудит-цепочка	Журнал доступа, где каждая запись содержит хеш предыдущей

---

5. Архитектура

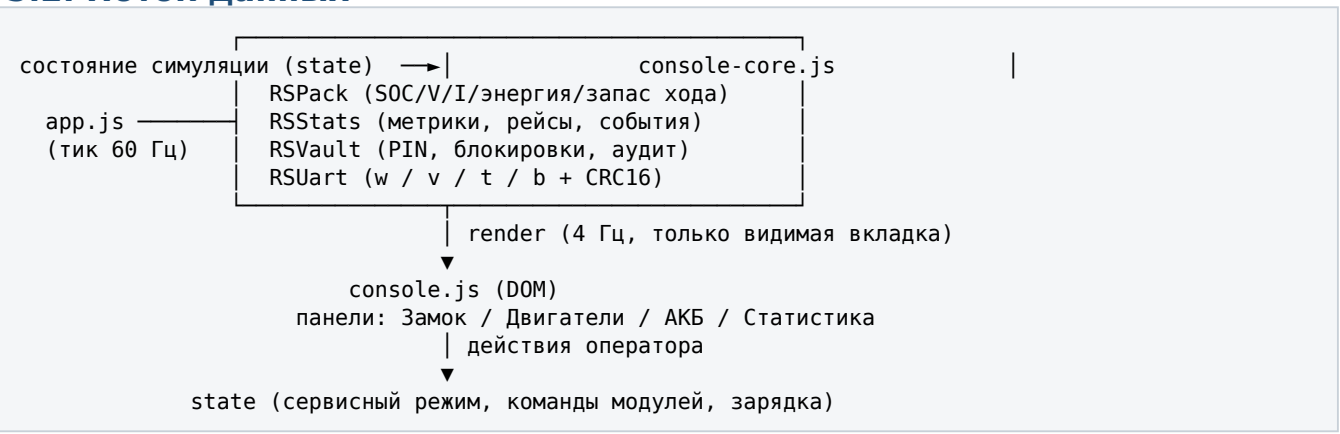
5.1. Слои и файлы

```
gui/
├── index.html        (+ вкладка «Сервис», панели 4 блоков, модальные окна)
├── styles.css        (+ стили сервисного пульта)
├── console-core.js   (НОВЫЙ) чистый слой логики без DOM:
│                       RSPack   — модель АКБ 12S3P LiFePO4
│                       RSUart   — строки протокола fw25 + CRC16/Modbus
│                       RSVault  — PIN-хранилище, блокировки, аудит-цепочка
│                       RSStats  — накопительная статистика, рейсы, экспорт
│                       RSFmt    — форматирование чисел/времени
├── console.js        (НОВЫЙ) DOM-слой окна «Сервис»: карточки модулей,
│                       клавиатура PIN, графики, таблицы, экспорт
├── app.js            (доработка) интеграция: модель АКБ, события,
│                       метрики сессии, сервисный режим, светофор
└── vision.js         (доработка) светофор в WORLD
```

Порядок загрузки скриптов (важно: `console-core.js` определяет `window.RS` раньше, чем `app.js` начинает им пользоваться):

```
<script src="console-core.js"></script>
<script src="vision.js"></script>
<script src="app.js"></script>
<script src="console.js"></script>
```

5.2. Поток данных



5.3. Принципы

1. **Разделение логики и представления.** `console-core.js` не знает о DOM, поэтому тестируется в Node без браузера.
2. **Обратная совместимость.** Все существующие элементы и сценарии (`app.js`, `vision.js`) сохраняются; новые функции подключаются опционально через `window.RS` с проверкой наличия.
3. **Безопасные интерлоки.** Любое ручное вмешательство возможно только вне авторежима и вне E-STOP; сервисный режим явно индицируется.
4. **Данные не теряются.** Статистика и аудит сбрасываются на диск (`localStorage`) периодически и по событию.

---

6. Модель данных

6.1. Ключи `localStorage`

Ключ	Содержимое	Версия схемы
`rus_slam_stats_v1`	`{version, updatedAt, totals, events, trips[], daily{}, session}`	1
`rus_slam_lock_v1`	`{version, salt, pinHash, fails, lockUntil, audit[], locked}`	1
`rus_slam_route_groups`	Существующие группы точек (не меняется)	—
`rus_slam_theme`	Существующая тема (не меняется)	—

6.2. Структура статистики

```
totals = {
  trips: 0,            // завершённых рейсов
  distanceM: 0,        // суммарный пробег, м
  energyWh: 0,         // суммарная энергия, Вт·ч
  movingSec: 0,        // время в движении, с
  cargoKg: 0,          // перевезено груза, кг
  cargoTrips: 0,       // рейсов с грузом
  maxSpeed: 0,         // максимум, м/с
  chargeCycles: 0,     // циклов заряда
  manualOps: 0,        // ручных команд двигателям
  obstacles: 0,        // объездов/обнаружений препятствий
  locks: { open: 0, denied: 0, changed: 0 },
  signs: { stop: 0, pedestrian_crossing: 0, speed_bump: 0 },
  lights: { red: 0, green: 0 },
  faults: 0            // аварий/сбоев миссии
};

trip = {
  id, startedAt, endedAt, durationSec, movingSec, distanceM, energyWh,
  cargoKg, avgSpeed, maxSpeed, from, to, result: "ok" | "aborted" | "fault"
};
```

6.3. Сессионные метрики (в памяти)

`session = { distanceM, energyWh, movingSec, maxSpeed, startedAt, whPerKm }` — интегрируются в `RSStats.tick(dt, sample)` с шагом кадра симуляции.

---

7. Алгоритмы

7.1. Модель АКБ 12S3P LiFePO4

Параметры (совпадают с `docs/TECHNICAL_SPECIFICATION.md`):

```
12S3P, банки 32700 (3.2 В / 6200 мА·ч), номинал 38.4 В,
диапазон 30.0…43.8 В, ёмкость 18.6 А·ч = 714.2 Вт·ч,
внутреннее сопротивление сборки ≈ 0.075 Ом, BMS 60 А, предохранитель 25 А
```

Кривая OCV (на элемент, кусочно-линейная):

U эл., В	2.50	2.80	3.00	3.10	3.20	3.25	3.30	3.33	3.35	3.40	3.45	3.65
SOC, %	0	3	7	10	15	30	50	70	85	95	98	100

- `socFromVoltage(U_пак)` — обратная интерполяция по кривой;
- `voltageFromSoc(soc)` — прямая интерполяция;
- терминальное напряжение под нагрузкой: `U = OCV − I·R`, при заряде `U = OCV + |I|·R`;

- разряд считается **по энергии**: `dE = U·I·dt/3600`, `dSOC = dE/714.2·100 %`;
- запас хода: `range = E_остаток / расход` где расход берётся из сессии (Вт·ч/км), при отсутствии данных — паспортные 20–25 км на полный заряд;
- пороги: 35.5 В — «ползучий» (SOC-индикация «низкий заряд»), 33.5 В — «критический» (гейт безопасности закрывается).

7.2. Ручная команда двигателю

```
1. Оператор вводит значение (угол, °) или ШИМ (%, −100…+100)
2. Валидация: диапазон угла ±180°, ШИМ ±100 % (в протоколе ±1000),
   шаг дискретности 0.0075° (133.33 шага/град), проверка интерлоков
3. Формирование строки: `w <XX> <угол>` для руля, `v <0…150> [f/b]` для тяги,
   `b [id]` для тормоза, `k`/`q`/`ph` для калибровок
4. Строка выводится в журнал; применяется к модели модуля
5. Обновление целевых значений: steer → target, pwm → тяга
6. Модель отрабатывает ограничения: скорость перекладки 360°/с,
   разгон тяги 2.6 (м/с)/с, ток по ШИМ, температура по нагрузке
7. Квитирование: если |ошибка| ≤ 0.5° — «в позиции»
```

Шаг 6 исполняется в симуляторе при сервисном режиме: угловой сервопривод сходится к цели с ограничением скорости, колесо разгоняется/тормозится, накапливается путь колеса (для контроля энкодера).

7.3. Замок ячейки (PIN)

```
Хранение:  борт (gui/backend.py): gui/state/lock.json —
           salt = random 16 байт (hex)
           pinHash = SHA-256(salt + ":" + pin)
           Пульт, открытый через борт (адрес /console того же сервера),
           работает с ЭТИМ замком по API: PIN и журнал общие с основным
           экраном. Локальное хранилище браузера включается только когда
           сервера нет (страница открыта как файл) — демо-режим.
Проверка:  attempts < 5 ? compare(hash(input)) : lockout 30 с
Успех:     fails = 0, lockUntil = 0, запись аудита {action:"unlock", ok:true}
Неудача:   fails++, запись аудита {action:"unlock", ok:false, fails}
Аудит:     каждая запись хранит prevHash; hash = SHA-256(prevHash + поля)
           → обнаружение правки/удаления записи (tamper-evident)
Смена PIN: требуется текущий PIN, новый 4–8 цифр, повтор;
           смена фиксируется в аудите и статистике
```

Автозакрытие: через 30 с после открытия, если груз снят/уложен (настраиваемый таймер), либо вручную.

7.4. Рейс и события

- **Рейс** начинается при переходе `state.auto: false → true` и завершается при `true → false` либо по прибытии на финальную точку/после сдачи груза; записываются дистанция, энергия, время, груз, средняя и максимальная скорость, результат.
- **События** инкрементируются в точках симуляции: знак (`stop`/`pedestrian_crossing`/`speed_bump`) — при срабатывании зоны, светофор — при остановке на красный и возобновлении на зелёный, препятствие — при срабатывании объезда/тупика, ручное вмешательство — при «Ввод» в карточке двигателя.

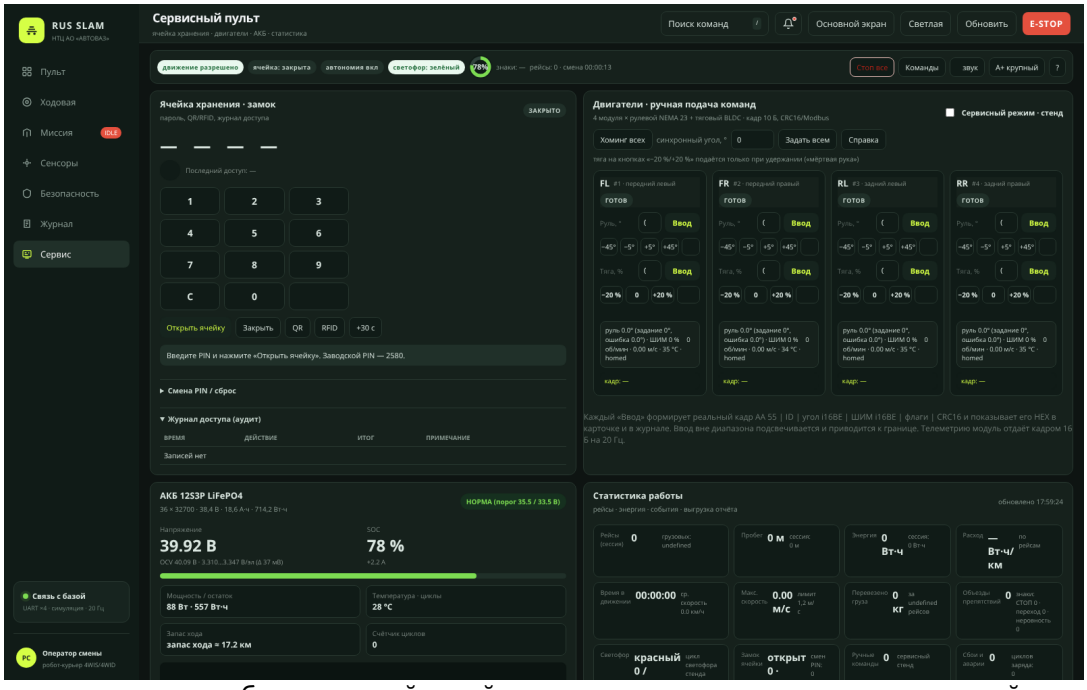
---

8. Интерфейс окна «Сервис»

```
┌──────────────────────────────────────────────────────────────────────────┐
│ СЕРВИС · ячейка хранения · двигатели · АКБ · статистика                  │
├──────────────────────────────────┬───────────────────────────────────────┤
│ 1. ЯЧЕЙКА ХРАНЕНИЯ               │ 3. ДВИГАТЕЛИ (FL, FR, RL, RR)         │
│  ● состояние: ЗАКРЫТО            │  ┌──────────────┐ ┌──────────────┐    │
│  [ •••• ] [Открыть]              │  │ FL           │ │ FR           │    │
│  клавиатура 1…9,0,⌫,C            │  │ руль  [ 45]° │ │ руль  [  0]° │    │
│  [QR] [RFID] [Сменить PIN]       │  │       [Ввод] │ │       [Ввод] │    │
│  попытки 1/5 · журнал доступа    │  │ тяга  [+20]% │ │ тяга  [  0]% │    │
├──────────────────────────────────┤  │       [Ввод] │ │       [Ввод] │    │
│ 2. АКБ 12S3P LiFePO4             │  │ факт: 44.8°  │ │ факт: 0.0°   │    │
│  ○ SOC 68% · 38.9 В · 12.4 А     │  └──────────────┘ └──────────────┘    │
│  остаток 412 Вт·ч · 14 км        │  ┌──────────────┐ ┌──────────────┐    │
│  ячейки 3.31…3.34 В (Δ 0.03)     │  │ RL           │ │ RR           │    │
│  модули: 38.9/38.8/38.9/38.8     │  └──────────────┘ └──────────────┘    │
│  уровень: НОРМА · история        │ [Стоп все] [Момент] [Хоминг] [Синхро] │
│                                  │ СЕРВИСНЫЙ РЕЖИМ · СТЕНД  [ Войти/Выйти ] │
├──────────────────────────────────┴───────────────────────────────────────┤
│ 4. СТАТИСТИКА                                                            │
│  [рейсы] [пробег] [энергия] [груз] [средняя] [запас] | события: знаки, светофор │
│  графики: энергия по рейсам · история SOC/скорости                       │
│  таблица последних рейсов     [CSV] [JSON] [Отчёт (печать)] [Сбросить]   │
└──────────────────────────────────────────────────────────────────────────┘
```

8.0. Как окно выглядит в работе

Скриншот снят с работающего сервера (`python3 gui/backend.py --port 8080 --source sim`), вкладка «Сервис», браузер 1600×1000:



Сверху — липкая панель быстрых действий: чипы «движение разрешено», «ячейка закрыта», «автономия вкл», фаза светофора, кольцо SOC и кнопки «Стоп все», «Команды», «звук», «A+ крупный», «?». Ниже — четыре панели: замок ячейки хранения с PIN-клавиатурой и журналом доступа, карточки четырёх модулей с полями угла и тяги, панель АКБ с напряжением/током/остатком и статистика рейсов.

8.1. Десять улучшений интерфейса и взаимодействия (реализованы)

№	Идея	Что сделано	Зачем оператору

| **И-1** | Панель быстрых действий | Липкая панель `#csl-toolbar` над окном: живые чипы состояния (движение/E-STOP, ячейка, автономия/стенд, светофор), кольцо SOC, сводка рейсов и времени смены, кнопки «Стоп все», «Команды», «звук», «A+ крупный», «?» | Главные статусы и аварийные действия всегда на виду. Открытие ячейки — только в панели «Ячейка хранения» (кнопка «Открыть ячейку», `Alt+L`): дубля кнопки в тулбаре нет, в тулбаре остался чип состояния ячейки |

| **И-2** | Палитра команд и горячие клавиши | `Ctrl+K` (или `/`) открывает палитру с нечётким поиском: вкладки, открытие ячейки, стенд, стоп, хоминг, экспорт CSV/JSON, печать отчёта, тема, звук, масштаб, E-STOP. Добавлены `Alt+1…7`, `Alt+L/S/X/Z/A`, `Esc` | Работа одной рукой, без мыши; обучение новичков через встроенную справку |

| **И-3** | Монитор смены в шапке | Кольцо SOC на canvas, время смены, число рейсов, фаза светофора, «последний знак» (из ленты событий) и обновление заголовка вкладки браузера (`[ячейка открыта · стенд] RUS SLAM …`) | Оператор видит состояние робота, даже когда вкладка не активна |

| **И-4** | Валидация полей ввода | Поля угла/тяги подсвечиваются при выходе из диапазона, автокламп на `blur` с пояснением в подсказке и в журнале; недоступные действия блокируются с объяснением причины | Исключены «молчаливые» ошибки ввода и нештатные кадры |

| **И-5** | «Мёртвая рука» для тяги | Кнопки ±20 % подают тягу только при удержании; отпускание, уход курсора, `blur`, скрытие вкладки или выход из режима «стенд» мгновенно отправляют нулевую ШИМ | Защита от «убежавшего» робота при стендовой отладке |

| **И-6** | Звуковая обратная связь | Web Audio: отдельные сигналы успеха, разблокировки замка, предупреждения, ошибки и «тик»; общий тумблер и выбор сохраняются в `localStorage` | Не нужно смотреть на экран, чтобы понять результат операции |

| **И-7** | Свои модальные окна | Замена `confirm/alert`: подтверждения (хоминг, сброс статистики), предпросмотр печатного отчёта, показ кадра UART, справка; закрытие по Esc, клику по фону и кнопке × | Единый стиль, предсказуемое поведение, доступность (`role=dialog`, фокус) |

| **И-8** | Журнал сервисных действий | Отдельная лента с фильтрами «Все / Успех / Внимание / Ошибки», поиском и копированием HEX-кадра одной кнопкой | Разбор инцидентов и сдача отчёта без чтения общей консоли |

| **И-9** | Крупный интерфейс и доступность | Режим «A+» (`body.ui-large`), `aria-live` для PIN/статусов, `aria-pressed` для тумблеров, `focus-visible`, `title`-подсказки с диапазонами; настройка сохраняется | Работа с планшета/в перчатках, соответствие требованиям доступности |

| **И-10** | Синхронизация между окнами | `BroadcastChannel` + событие `storage`: открытие/закрытие ячейки одним окном отражается в другом, обновление статистики после экспорта; индикатор «второе окно пульта активно» | Исключены расхождения при работающем рядом пульте и втором операторе |

Дополнительно: кнопка **«+30 с»** откладывает автозакрытие ячейки, а кольцо автозакрытия в панели замка показывает остаток времени.

Клавиатурные сокращения окна (когда открыта вкладка «Сервис» и фокус не в текстовом поле): `1…0` — цифры PIN, `Backspace/Delete` — стереть цифру, `Enter` — «Открыть ячейку», `Esc/C` — очистить ввод. Общие: `Ctrl+K//` — палитра команд, `Alt+1…7` — вкладки, `Alt+L` — ячейка, `Alt+S` — стенд, `Alt+X` — стоп всех, `Alt+Z` — звук, `Alt+A` — масштаб. Движение (`WASD`) при открытой вкладке продолжает работать, если сервисный режим выключен.

---

9. Интеграция с существующим пультом

Точка	Изменение (реализовано)
`state` (app.js)	добавлены: `serviceStand`, `lockOpen`, `origin`, `_wasSpin`; модули получили поля `svcSteer`/`svcPwm`/`enabled`
Модель АКБ	`RS.pack`: 12S3P LiFePO4 (30…43,8 В) вместо прежней линейной модели 44–54,6 В; разряд по энергии `dE = U·I·dt/3600`
Светофор	в `WORLD.signs` добавлен `kind:"light"`; цикл 6 с зелёный / 1,5 с жёлтый / 5 с красный, остановка авторежима в зоне 2,4 м, отрисовка на карте (`vision.js`)
`followRoute()`, `applyKeys()`	при `state.serviceStand` скорости обнуляются, автономия и телеуправление заблокированы
`sim()`	вызов `RSConsole.onSimTick(dt, state, …)` → `RS.StatsStore.tick`; обсчёт светофора; счёт объездов по фронту `spinClear`; `RSConsole.noteSign()` в зонах знаков
Интерлоки	старт миссии и «Пуск» программы запрещены при открытой ячейке; «стенд» запрещает запуск программы
Тумблер «Замок отсека»	синхронизирован с замком окна «Сервис» (`closeLock`)
`loop()`	вызов `RSConsole.frame(now)` — перерисовка окна ≤ 4 Гц только для видимой вкладки

Обратная совместимость: если `console-core.js` не загружен, `app.js` работает по прежней логике (все вызовы через `window.RS &&`).

---

10. Безопасность и отказоустойчивость

Риск	Мера
Подбор PIN	лимит 5 попыток, блокировка 30 с, аудит, хеш вместо открытого PIN
Подделка журнала доступа	хеш-цепочка записей (изменение любой записи ломает цепочку)
Случайное движение при отладке	сервисный режим блокирует авто, миссию и телеуправление
Ввод команды при аварии	интерлок E-STOP и авторежима
Выход за пределы механики	валидация диапазонов (угол ±180°, ШИМ ±1000), предупреждения
Падение `localStorage`	`try/catch`, деградация до памяти процесса
Потеря статистики	запись раз в 15 с, при завершении рейса и `beforeunload`
Перерисовка тяжёлых графиков	частота 4 Гц, только для видимой вкладки

---

11. Соответствие критериям регламента (задание № 2)

| Критерий | Балл | Что даёт окно «Сервис» |

| :--- | :-: | :--- |

| Автономное движение по маршруту | 7 | Статистика рейсов и аварий — подтверждение автономности и работы с нештатными ситуациями |

| Перемещение груза | 5 | Замок ячейки, контроль груза, запрет открытия вне площадки |

| Программно-аппаратный комплекс | 5 | Полный контур: от ввода задания до ручного сервиса модулей |

| Безопасное перемещение | 5 | Индикация порогов АКБ, интерлоки, лог событий |

| Безопасность груза | 5 | PIN/QR/RFID, аудит доступа, защита от подделки журнала |

| Информирование пользователя | 5 | Понятные показатели, графики, печатный отчёт |

| Сбор, хранение и управление данными | 5 | Персистентная статистика, экспорт CSV/JSON, отчёт, аудит |

---

12. Тест-план

12.1. Модульные тесты (`gui/tests/console.test.js`, Node, без зависимостей)

| № | Проверка |

| :-: | :--- |

| 1 | CRC16/Modbus: эталонный вектор `"123456789"` → `0x4B37` (совпадает с парсером Zapret) |

| 2 | Команда: адресация по роли, десятичный угол, v 0…150, CRC16 строки |

| 3 | Клампы: угол ±32767 cdeg, ШИМ ±1000 |

| 4 | Телеметрия: разбор `@F` (20 полей) и `@TLM` (ключ=значение), CRC, `sc_rpm`, `deg` |

| 5 | Модель АКБ: `socFromVoltage(43.8)=100`, `socFromVoltage(30.0)=0`, монотонность, обратимость, пороги |

| 6 | Энергия: интегрирование `dE=U·I·dt`, падение SOC, запас хода |

| 7 | PIN: заводской PIN, неверный PIN, блокировка после 5 попыток, разблокировка по таймеру, смена PIN |

| 8 | Аудит: цепочка хешей валидна; правка записи обнаруживается |

| 9 | Статистика: добавление рейсов, агрегаты, CSV, JSON, сброс |

| 10 | Метрики сессии: пробег/энергия/время в движении, расход Вт·ч/км |

12.2. Интеграционные DOM-тесты (`gui/tests/jsdom.console.test.js`, 93 проверки)

Прогон поднимает настоящий `gui/index.html` в jsdom, подключает скрипты в порядке из разметки и проверяет: загрузку без исключений; вкладку «Сервис» и её панели; замок (пустой PIN, неверный PIN, PIN 2580 → «ОТКРЫТО», синхронизацию с `state.cargoLock`, аудит, закрытие, ввод PIN с физической клавиатуры); интерлок «стенд» до ручной подачи; строки `w FL 20.0/v 40 f` после «Ввода»; валидацию полей с автоклампом; «мёртвую руку»; панель быстрых действий; палитру команд; звуковые настройки; модальные окна (подтверждение, предпросмотр отчёта, справка); журнал сервисных действий (фильтры, поиск, копирование кадра); синхронизацию окон через `storage`; неизменность корпуса в стенде; рейс и экспорт; цикл светофора (красный/зелёный); запрет старта миссии и выдачи груза без PIN.

```
# jsdom нужен только для этого прогона и в зависимости проекта не входит
npm --prefix /tmp/harness i jsdom           # один раз
NODE_PATH=/tmp/harness/node_modules node gui/tests/jsdom.console.test.js
```

12.3. Ручной чек-лист

1. `python3 -m http.server 8080` из `gui/`, открыть в браузере;
2. проверить окно в светлой и тёмной темах;
3. выполнить рейс А→Б, убедиться, что рейс попал в статистику;
4. выгрузить CSV и JSON, открыть печатный отчёт;
5. очистить `localStorage` — убедиться в корректной инициализации.

12.4. Критерии приёмки

- все модульные тесты проходят (`node gui/tests/console.test.js`);
- интеграционные DOM-тесты — без ошибок (`jsdom.console.test.js`);
- существующие вкладки пульта работают без регрессий;
- PDF-документация пересобрана (`tools/pdf/md2pdf.py --check` — успех);
- сценарий С-1 (получение груза по PIN) воспроизводится в симуляторе.

12.5. Результаты прогонов (2026-10-02)

| Прогон | Команда | Итог |

| :--- | :--- | :-: |

| Ядро (без зависимостей) | `node gui/tests/console.test.js` | **32 passed, 0 failed** |

| Интеграция DOM (jsdom 24) | `NODE_PATH=/tmp/harness/node_modules node gui/tests/jsdom.console.test.js` | **93 passed, 0 failed** |

Покрытые группы: CRC16 и кадры протокола; клампы; OCV-кривая и энергобаланс; пороги АКБ; PIN (неверный ввод, блокировка, смена, аудит, события); статистика (метрики, рейсы, прерывание по E-STOP, события, персистентность, сброс, экспорт).

---

13. Ограничения и развитие

Ограничения текущей ревизии:

- GUI остаётся автономным симулятором; связь с ROS 2 — через `rosbridge_server` (см. `GUI.md` §9);
- электрический замок отсека — модель (в железе требуется реле/соленоид, управляемый GPIO бортового ПК, см. `pcb/CONNECTOR_PINOUTS.csv`);
- PIN/QR/RFID — программная имитация; в железе: RC522 (SPI) и камера QR.

Дорожная карта:

1. Мост `rosbridge`: топики `/cargo/lock` (`std_msgs/Bool`), `/battery` (`rus_slam_interfaces/BatteryInfo`), `/modules/state` — вместо модели;
2. реальный QR-сканер (камера) и RFID RC522 с белым списком меток;
3. экспорт отчётов в PDF на стороне бортового ПК (текущий — печать браузера);
4. журнал статистики в SQLite/бортовом хранилище с синхронизацией на сервер диспетчера;
5. роли операторов (сменный инженер/получатель) с разными правами.

---

14. Порядок работ (выполнено)

| Этап | Содержание | Артефакт |

| :-: | :--- | :--- |

| 1 | Технический план и ТЗ | `docs/SERVICE_CONSOLE.md` (этот документ) |

| 2 | Слой логики | `gui/console-core.js` |

| 3 | Разметка и стили окна | `gui/index.html`, `gui/styles.css` |

| 4 | DOM-слой окна | `gui/console.js` |

| 5 | Интеграция с симулятором | `gui/app.js`, `gui/vision.js` |

| 6 | Тесты | `gui/tests/console.test.js` (32/32) + `gui/tests/jsdom.console.test.js` (93/93) |

| 7 | Документация | `docs/GUI.md`, `README.md`, `gui/README.md`, PDF-копии |

| 8 | Десять улучшений интерфейса (И-1…И-10) | §8.1 этого документа, gui/console.js, gui/index.html, gui/styles.css |

---

15. Лицензия и авторы

- Лицензия: MIT (см. README.md);
- Команда RUS SLAM, НТЦ АО «АВТОВАЗ», конкурс «Первые в профессии», задание № 2 «Автономное роботизированное устройство для внутренней логистики НТЦ АО „АВТОВАЗ“».